

Foundations of Data Science for Biologists

# Introduction to Python

BIO 724D

2026-FEB-24

Instructors: Greg Wray and Kayla Wilhoit

# Learning goals for today

Become familiar with Jupyter notebooks

Become familiar with the elements of Python syntax

Learn how Python differs from R: uses and syntax

-----

Write a program to model and visualize Boolean networks

# What is Python?



# What is Python?

Python is a **general programming language** (GPL)

Designed to be as **versatile** as possible

Differs from R and SQL, which are designed for **specific use cases**



## Features of Python

**Free, open-source, and cross-platform**

**Very-high-level-language** (VHLL) that simplifies access to a very large problem domain

Widely considered one of the **clearest** and **fastest** languages for programming

Extensive **libraries** for generalized and specific applications

Massive **community** providing support for learning and problem-solving

**Future-proof**: more modern than R and much more widely deployed



# Origins of Python

Python was created in the late 1980s by **Guido van Rossum**, a Dutch computer scientist

Goals: readable and versatile

Multi-paradigm: structured, object-oriented, functional, aspect-oriented

Named for **Monty Python's Flying Circus**

British surreal sketch comedy troupe (1969-1974)

As of 2026:

Among top

Widely adopted

Dominates



last six+ years

science, commerce

intelligence, automation



Guido at PyCon 2024



# Transitioning to a new language

**The good news:** your experience with Unix, SQL, and *especially* R will help (a lot!)

Basic programming concepts remain the same

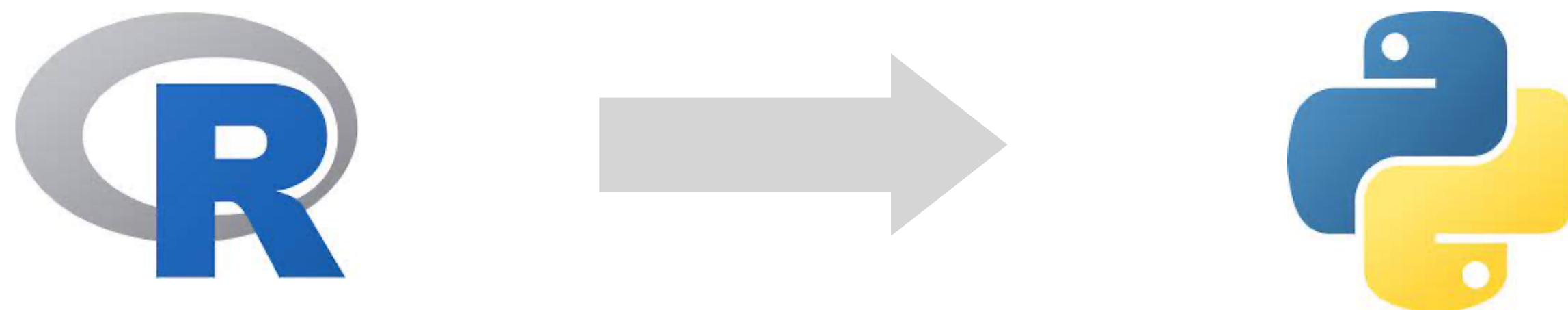
Python is more similar to R than many other languages (e.g., C, Java, Rust)

**The challenge:** some key differences from R can create unexpected errors and results

**Today's goals:**

Get you working with Python code as quickly as possible

Minimize the frustration caused by **differences** between R and Python



# Introduction to Jupyter Notebooks

# Introducing JupyterLab

**Jupyter Lab** is an interactive notebook environment

Provides two kinds of cells:

- Text** that renders, with markdown

- Code** that runs, with color formatting

Preserves output, including text, graphs, and images



Practical features:

- Runs in a browser window

- Supports auto-complete (tab)

- Files are plain-text with extension `.ipynb`

- Because Jupyter Lab is web-based, it can't be designated as the default app



# The JupyterLab interface: file operations

new folder

upload file

save file

close tab

other tabs

new tab

file browser

path to pwd

files in pwd

current tab

The screenshot shows the JupyterLab interface with a file browser on the left and a code editor on the right. The file browser displays the current working directory and a list of files. The code editor is currently empty. The interface includes a menu bar at the top with options: File, Edit, View, Run, Kernel, Tabs, Settings, and Help. The file browser has a search bar and a table of files. The code editor has a status bar at the bottom showing the current mode, line and column numbers, and the file name.

| Name                 | Modified |
|----------------------|----------|
| 21_Notes_Py_Da...    | 11mo ago |
| 21_py_datatypes...   | 11mo ago |
| 21_py_datatypes...   | 11mo ago |
| 21_Py_datatypes...   | 11mo ago |
| covid-ref.fasta      | 11mo ago |
| nytimes-covid-d...   | 11mo ago |
| parsing_files_ex...  | 11mo ago |
| Spike-protein-ali... | 11mo ago |
| Untitled.ipynb       | 11mo ago |

Mode: Command | Ln 1, Col 1 | parsing\_files\_example.ipynb | 1

# The JupyterLab interface: block operations

current

run current; select next

move up/down

new above/below

delete

text block

code block

output

The screenshot shows the JupyterLab interface with the following components and annotations:

- File Explorer (Left):** A sidebar showing a file tree. The file `23_py_numpy_p...` is selected. A blue arrow labeled "current" points to this file.
- Code Editor (Center):** The main workspace showing a notebook with two code blocks.
  - Code Block 1:** Contains a text block explaining multidimensional array indexing, followed by a code cell [52] that creates a 3D array and prints it. A blue arrow labeled "run current; select next" points to the "Run" button (a play icon) in the toolbar above the code cell.
  - Code Block 2:** Contains a code cell [53] that retrieves a single element from the 3D array.
- Toolbar (Right):** A vertical toolbar on the right side of the code editor with icons for block operations. Blue arrows point to these icons:
  - "move up/down": Points to the up and down arrow icons.
  - "new above/below": Points to the plus and minus icons.
  - "delete": Points to the trash can icon.
- Output (Bottom):** The output of the code cells is displayed at the bottom of the editor. A blue arrow labeled "output" points to this area.

## Reference: quick orientation to the JupyterLab interface

To **select a cell**: click to its left *or* directly on the gray box

The **current cell** is indicated by a blue bar at the left

To **run/update** a cell: select it, then press the play icon or press **shift-return**

To **edit code** in a cell: select it, then click to position the cursor; you can now type

To **add, move, or delete** a cell: select it, then use the icons at the upper right

To **change the type** of cell (code vs markdown), select it and use the drop-down menu

The **file browser** shows the pwd and list of files; the active file is highlighted in blue



# First Python program



# Quick start: Python syntax for R programmers

# 1 Assignment and operators

Basic assignment works similarly to R

Data types are inferred from values (*dynamic typing*)

Variable names can be re-assigned to new values / types (*re-binding*)

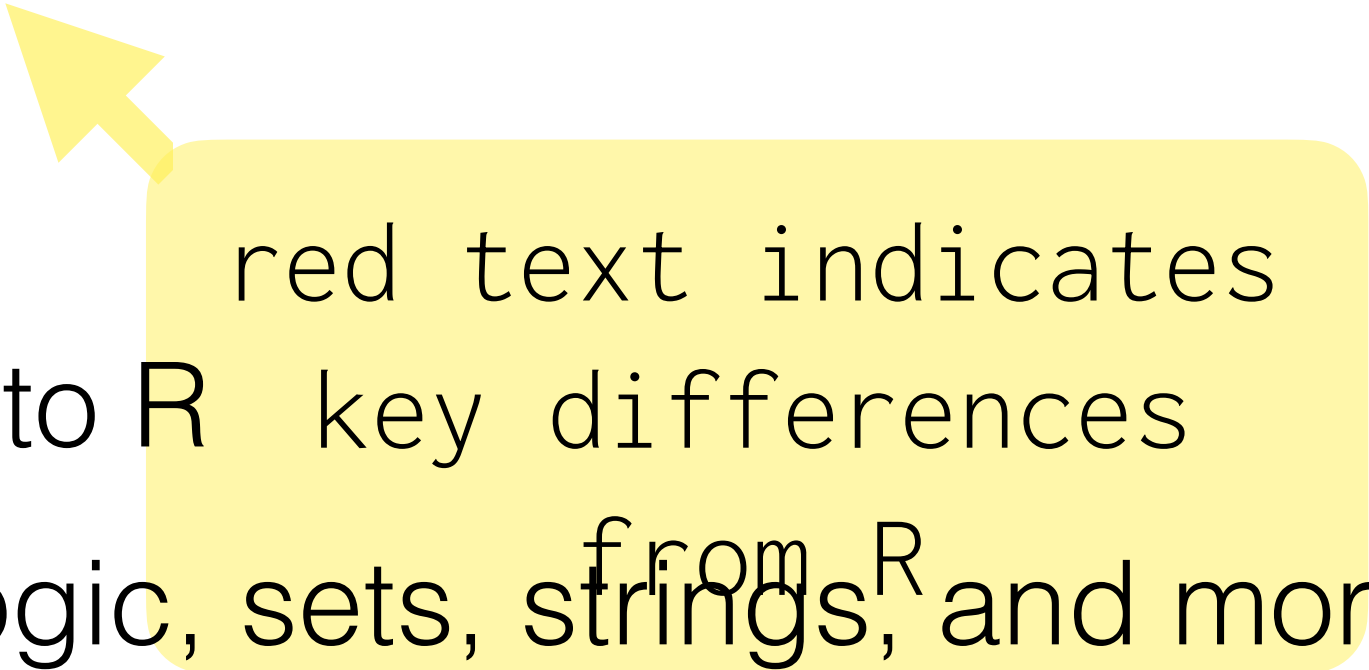
Data types can be converted where sensible (called *casting* in Python)

Main difference: use the = operator

Most common **operators** work similarly to R

Includes operators for arithmetic, logic, sets, strings, and more

Main difference: use \*\* for exponentiation, not ^



red text indicates  
key differences  
from R

## 2 Data types and data structures

### Data types:

`int`, `float`, `bool`, `complex` hold the same kinds of values as in R

However, they hold **single values**, not vectors of values

`str` is a **vector of characters**

### Data structures:

`list` in Python is similar to R

`tuple`, `set`, `dictionary` are **common** data structures in Python, but not part of R

Other data structures are also common, including arrays and data frames

In general, Python programs use a **wider range** of data structures than R



### 3 Iterable data objects

*Iterable* means that a data object can return one value or item at a time

In Python, only **containers and strings** are iterable (**list**, **tuple**, **set**, **str**, etc.)

Unlike R, where nearly all data types are iterable (basic types are **vectors**)

Most Python iterables can hold a **mix of data types**

Unlike R, where only lists can contain items of different types

The exception in Python is **str**, which only holds characters



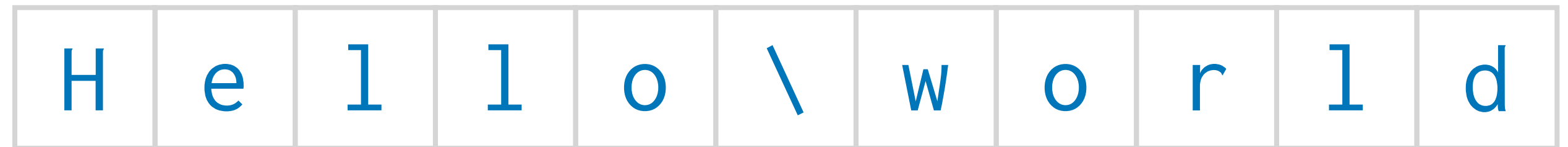
## 3.1 Strings in Python

Strings are **vectors of characters** in Python

Strings are indexable and iterable in both Python and R

However, in Python indexing refers to individual characters, not items

```
py_string = 'Hello world'
```



```
r_string <- 'Hello world'
```

|             |
|-------------|
| Hello\world |
|-------------|

```
py_string[0]
```

returns 'H'

```
r_string[1]
```

returns 'Hello world'

## 4 Indexing

Python uses **square-bracket indexing** for most iterables

This works similarly to indexing in R, with **two main differences** (next slide)

Dictionaries use **key indexing** (dictionaries don't exist in R)

# 4.1 Square-bracket indexing

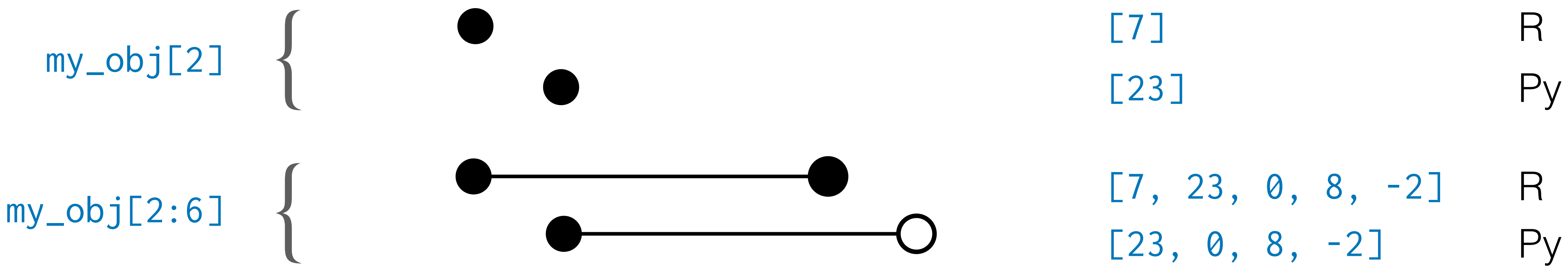
Square-bracket indexing in Python is different from R in two ways:

**Zero-based** (first object has address 0 rather than 1)

Slices **do not include the end**

|        |    |   |    |   |   |    |   |   |                |
|--------|----|---|----|---|---|----|---|---|----------------|
| my_obj | 12 | 7 | 23 | 0 | 8 | -2 | 4 | 2 | ← values       |
|        | 1  | 2 | 3  | 4 | 5 | 6  | 7 | 8 | ← R index      |
|        | 0  | 1 | 2  | 3 | 4 | 5  | 6 | 7 | ← Python index |

Examples:





## 4.2 Examples of indexing in Python

|        |    |   |    |   |   |    |   |   |          |
|--------|----|---|----|---|---|----|---|---|----------|
| my_obj | 12 | 7 | 23 | 0 | 8 | -2 | 4 | 2 | ← values |
|        | 0  | 1 | 2  | 3 | 4 | 5  | 6 | 7 | ← index  |

|                 |                             |                             |
|-----------------|-----------------------------|-----------------------------|
| my_obj[1]       | 7                           | single item                 |
| my_obj[4:5]     | [8]                         | slice of length 1           |
| my_obj[3:]      | [0, 8, -2, 4, 2]            | open slice to end           |
| my_obj[:2]      | [12, 7]                     | open slice from start       |
| my_obj[:]       | [12, 7, 23, 0, 8, -2, 4, 2] | all items                   |
| my_obj[-1]      | 2                           | last item*                  |
| my_obj[0::3]    | [12, 0, 4]                  | slice with step = 3*        |
| my_obj[-1:1:-1] | [2, 4, -2, 8, 0, 23]        | slice with reverse step*    |
| my_obj[::-1]    | [2, 4, -2, 8, 0, 23, 7, 12] | reverse* (“Martian smiley”) |

## 5 Mutable and immutable objects

Python distinguishes between **mutable** and immutable objects

Mutable data objects can be modified, immutable objects cannot

R does **not support immutability**

Immutable data objects occupy less memory and provide faster performance

Immutable objects *can* be deleted

Re-binding the name of an immutable object erases the original, creates a new object



## 6 Syntax and formatting

Python was designed to have very clear syntax

Expressions often contain fewer characters than other programming languages

Expressions mimic statements in English as much as possible

Indenting is required to indicate code blocks

Best practice is to use **4 spaces**

Any number of tabs or space is allowed, but must be consistent

Indent again to indicate nested code blocks (no limit to nesting)

## 6.1 Control flow

Python provides `if/else`, `for`, and `while` structures that are similar to R

Python provides a few `additional structures and options` not available in R

Test conditions are expressed similarly to R

Main difference concerns logical operators: use `and`, `or`, `not` instead of `&`, `|`, `!`

To define: use `colon and indenting` rather than curly braces to define the code block

```
if my_number % 2 == 0:
    print("even number")
else:
    print("odd number")
<next statement>
```

```
for i in my_iterable:
    print(i, "squared is", i**2)
<next statement>
```

## 6.2 Functions

Python has **two** representations of functions

**Standard function:** works much like a function in R

To use: `function(arguments)`

**Method:** a function for a **specific data class**; not implemented in R

To use: `data_object.function(arguments)`

Methods can be connected into pipes

To define: use **colon and indenting** rather than curly braces to define the code block

```
def clean_up(input_text):  
    return input_text.upper().trim()  
<next statement>
```



## 7 Libraries

A vast constellation of useful libraries is available for Python (as for R)

**Standard modules:** built and maintained by the Python Software Organization

Extensive and provide many core functions and data classes (“batteries included”)

Do not need to be installed: distributed with Python installation and updates

**Third-party packages:** built and maintained by others (like packages in R)

Quality, reliability, and compatibility with updates varies enormously

Some are widely used and reliable (e.g., SciPy, NumPy, Matplotlib, Pandas)

Packages need to be installed before use and manually updated

## 7.1 Working with libraries

To **access** a library, import it within your program (**syntax** different from R)

```
import csv
import numpy as np                # aliases are commonly used
from matplotlib import pyplot as plt  # import just part of large library
```

To **use** a function in a library, **prepend the name the library** (different from R)

```
import math                      # factorial() function is included
my_result = factorial(3)         # generates a NameError
my_result = math.factorial(3)    # computes 3! (3 factorial, not excited)
```

# Resources for Python



# Resources for learning Python

## Textbooks

Downey (2015) *Think Python* (2ed). Green Tea Press: [here](#)

Matthes (2023) *Python Crash Course* (3ed). No Starch Press. (for purchase)

Ramalho (2022) *Fluent Python* (2ed). O'Reilly: [here](#). (intermediate level)

McKinney (2022) *Python for Data Analysis* (3ed). O'Reilly: [here](#) (NumPy, Pandas)

## Tutorial

Python official tutorial: [here](#)

Software Carpentry: [here](#)

Note: links point to free access (legal); next slide as well

# Resources for Python programming

## Official Python documentation

The Python Language Reference: [here](#)

The Python Standard Library: [here](#)

## General Python programming reference

Martelli et al. (2017) *Python in a Nutshell* (3ed). O'Reilly

## Glossaries of Python terminology

Official Python Glossary: [here](#)

Python Lingo from Fluent Python: [here](#)

## Python style guide

PEP 8: [here](#)

